SkyChat API Documentation — Mobile App Integration Guide

> **Base URL:** `https://nd1cgptf-3002.uks1.devtunnels.ms`
>
> **Socket.IO Server:** same URL (default namespace `/`)

Table of Contents

---

1. Architecture Overview

Layer	Technology
Backend	Node.js + Express 5 + TypeScript
Real-time	Socket.IO
Database	MongoDB Atlas (Mongoose ODM)
File Storage	Cloudinary
Architecture Pattern	Clean Architecture (Domain / Application / Infrastructure / Presentation)

Key design decisions your Flutter app must follow:

Decision	Detail
Messages are saved server-side	When you emit `chatMessage` via Socket.IO, the server auto-saves to MongoDB. Do NOT call REST to save messages.
Files are two-step	Step 1: Upload via REST `/upload-file` → get URL. Step 2: Emit `chatMessage` via Socket.IO with file metadata.
User is auto-created	`POST /go-to-room` creates the user if they don't exist. Always call this first.
`userId` can be `null`	Race condition — if a user disconnects before the message DB write completes, `userId` will be `null`. Handle gracefully.
No `video` type support	The current upload endpoint maps `video/*` MIME types to `"file"` (not `"video"`).

---

2. REST API Endpoints

2.1 Join / Create Room

Creates a user + room pair or returns the existing one.

```
POST {{base_url}}/go-to-room
Content-Type: application/json

{
  "name": "string",
  "room": "string"
}
```

Field	Type	Required	Description

`name` `string` ✅		Username (unique per room)
`room` `string` ✅		Room/channel name

Response `200` — New user created:

```
{
  "message": "done, user added",
  "data": {
    "id": "664a1e8fbe67237831a2c738",
    "name": "john",
    "room": "General",
    "isOnline": false
  }
}
```

Response `201` — User already exists:

```
{
  "message": "user existed",
  "data": {
    "id": "664a1e8fbe67237831a2c738",
    "name": "john",
    "room": "General",
    "isOnline": true
  }
}
```

Flutter example:

```
Future<User> joinRoom(String name, String room) async {
  final response = await http.post(
    Uri.parse('$baseUrl/go-to-room'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'name': name, 'room': room}),
  );
  final body = jsonDecode(response.body);
  return User.fromJson(body['data']);
}
```

2.2 Get Room Messages

Returns all persisted messages for a room, sorted oldest-first. Each message's `userId` is a **populated user object** (not just an ID).

```
GET {{base_url}}/all-messages-for-room?room={room_name}
```

Query Param	Type	Required	Description
`room`	`string`	✅	Room name

Response `200`:

```
{
  "message": "chat found",
  "data": [
    {
      "_id": "664a1e904d67237831a2c738",
      "content": "Hello!",
      "room": "General",
      "userId": {
        "_id": "664a1e8fbe67237831a2c738",
        "name": "john",
        "room": "General",
        "isOnline": true
      },
      "file": {
        "url": "https://res.cloudinary.com/.../image/upload/v123/file.png",
        "publicId": "skychat/abc123",
        "type": "image",
        "name": "photo.png"
      },
      "createdAt": "2026-06-04T09:12:06.885Z",
      "updatedAt": "2026-06-04T09:12:06.885Z"
    }
  ]
}
```


> ⚠ **Edge case:** `userId` may be `null` — see Section 6.

Use this endpoint to load chat history when the user enters a room.

2.3 Upload File

Uploads a file to Cloudinary and returns the URL + metadata. After uploading, emit the file metadata via Socket.IO `chatMessage`.

```
POST {{base_url}}/upload-file
Content-Type: multipart/form-data

file: (binary data)
```

Form Field	Type	Required	Description
`file`	binary	☑	The file to upload

Response `200`:


```
{
  "url": "https://res.cloudinary.com/dwld0gbaj/image/upload/v123/skychat/abc.png",
  "publicId": "skychat/abc123",
  "type": "image",
  "name": "original_filename.png"
}
```


`type` mapping rules:

Detected MIME	Returned `type`
`image/*`	`"image"`
`audio/*`	`"audio"`
`video/*`	**`"file"`** (⚠ not `"video"`)
everything else	`"file"`

Response `400` — No file:


```
{ "message": "no file uploaded" }
```


Response `500` — Upload error:


```
{ "message": "upload failed" }
```


Flutter example:

```
Future<FileAttachment> uploadFile(File file) async {
  final request = http.MultipartRequest(
    'POST',
    Uri.parse('$baseUrl/upload-file'),
  );
  request.files.add(await http.MultipartFile.fromPath('file', file.path));
  final streamedResponse = await request.send();
  final response = await http.Response.fromStream(streamedResponse);
  final body = jsonDecode(response.body);
  return FileAttachment.fromJson(body);
}
```

---

3. Socket.IO Events

Package (Dart): `socket_io_client` (https://pub.dev/packages/socket_io_client)

```
import 'package:socket_io_client/socket_io_client.dart' as IO;

IO.Socket socket = IO.io('https://nd1cgptf-3002.uks1.devtunnels.ms', <String, dynamic>{
  'transports': ['websocket'],
  'autoConnect': true,
});
```

3.1 Client → Server

`joinRoom`

Emitted right after socket connection to register in a room. The server sets `isOnline: true` and notifies others.

```
socket.emit('joinRoom', {
  'userName': 'john',
  'room': 'General',
});
```

Field	Type	Description
`userName`	`string`	Must match the name used in `POST /go-to-room`
`room`	`string`	Must match the room used in `POST /go-to-room`

`chatMessage`

Emitted to send a text message or a file message. The server auto-saves to MongoDB and broadcasts to the room.

Plain text:

```
socket.emit('chatMessage', {
  'text': 'Hello everyone!',
  'file': null,
});
```

With file attachment (must upload first via REST):

```
socket.emit('chatMessage', {
  'text': 'Check this out',
  'file': {
    'url': 'https://res.cloudinary.com/...',
    'publicId': 'skychat/abc123',
    'type': 'image',
    'name': 'photo.jpg',
  },
});
```

Field	Type	Required	Description
`text`	`string`	✅	Message text (can be empty `""` if file only)
`file`	`object`	❌	`null` or `FileAttachment` object

`typing`

Broadcasts typing status to other users in the same room.

```
// User started typing
socket.emit('typing', true);

// User stopped typing
socket.emit('typing', false);
```

Value	Type	Description
`true`	`boolean`	User is typing
`false`	`boolean`	User stopped typing

3.2 Server → Client

`message`

Received when any message is sent in the room (including your own).

```
socket.on('message', (data) {
  final userName = data['userName'] as String;
  final text     = data['text'] as String;
  final time     = data['time'] as String;       // "HH:MM" format
  final file     = data['file'] as Map?;          // null or FileAttachment
});
```

Field	Type	Description
`userName`	`string`	Sender name, or `"Chat"` for system messages
`text`	`string`	Message content
`time`	`string`	Formatted time (e.g. `"09:12"`)
`file`	`object?`	`null` or `FileAttachment` object

System messages (join/leave/welcome) have `userName: "Chat"`. Display them differently:

```
if (data['userName'] == 'Chat') {
  // Centered, muted, italic styling
} else if (data['userName'] == currentUserName) {
  // Right-aligned, own bubble
} else {
  // Left-aligned, other bubble
}
```

`joinedRoom`

Sent to the connecting user after a successful room join.

```
socket.on('joinedRoom', (data) {
  final room = data['room'] as String;
  // Now safe to fetch message history
});
```

`roomUsers`

Received whenever the user list changes (someone joined or left).

```
socket.on('roomUsers', (data) {
  final room  = data['room'] as String;
  final users = data['users'] as List;

  for (var u in users) {
    final id       = u['id'] as String;
    final name     = u['name'] as String;
    final room     = u['room'] as String;
    final isOnline = u['isOnline'] as bool;
  }
});
```

Field	Type	Description

`id`	`string`	User ID
`name`	`string`	Username
`room`	`string`	Current room
`isOnline`	`bool`	Online/offline status

`displayTyping`

Received when another user in the room starts or stops typing.

```
socket.on('displayTyping', (data) {
  final userName = data['userName'] as String;
  final isTyping = data['isTyping'] as bool;

  // Show/hide typing indicator
  if (isTyping) {
    showTypingIndicator(userName);
  } else {
    hideTypingIndicator(userName);
  }
});
```

---

4. Data Models

User

```
{
  "id": "664a1e8fbe67237831a2c738",
  "name": "john",
  "room": "General",
  "isOnline": true
}
```

Flutter:

```
class User {
  final String id;
  final String name;
  final String room;
  final bool isOnline;

  User({required this.id, required this.name, required this.room, required this.isOnline});

  factory User.fromJson(Map<String, dynamic> json) => User(
    id: json['id'] ?? json['_id'],
    name: json['name'],
    room: json['room'],
    isOnline: json['isOnline'] ?? false,
  );
}
```

FileAttachment

```
{
  "url": "https://res.cloudinary.com/.../image/upload/v123/skychat/abc.png",
  "publicId": "skychat/abc123",
  "type": "image",
  "name": "photo.png"
}
```

Field	Type	Values
`type`	`string`	`"image"`, `"audio"`, `"file"` (⚠ never `"video"`)

Flutter:

```dart
class FileAttachment {
  final String url;
  final String publicId;
  final String type; // "image" | "audio" | "file"
  final String name;

  FileAttachment({required this.url, required this.publicId, required this.type, required this.name});

  factory FileAttachment.fromJson(Map<String, dynamic> json) => FileAttachment(
    url: json['url'],
    publicId: json['publicId'],
    type: json['type'],
    name: json['name'],
  );

  bool get isImage => type == 'image';
  bool get isAudio => type == 'audio';
  bool get isFile => type == 'file';
}
```

Message

```
{
  "id": "664a1e904d67237831a2c738",
  "content": "Hello!",
  "room": "General",
  "userId": "664a1e8fbe67237831a2c738",
  "file": { ... },
  "createdAt": "2026-06-04T09:12:06.885Z"
}
```

> `userId` is a **string** when returned from `createMessage` / `SaveMessageUseCase`, but a **populated User object** when returned from `GET /all-messages-for-room`.

Flutter:

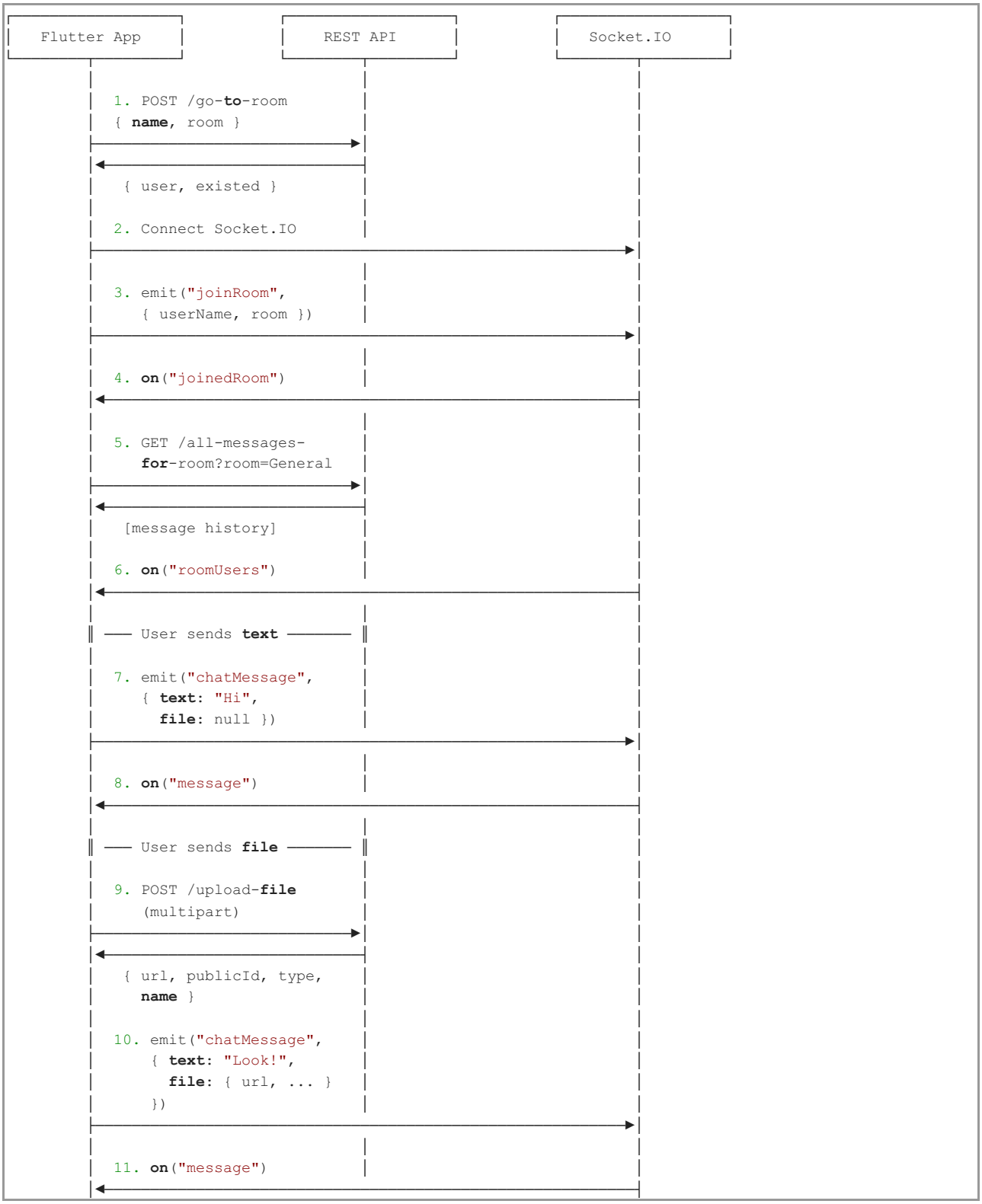
```dart
class ChatMessage {
  final String id;
  final String content;
  final String room;
  final String userId;
  final String? userName;  // extracted from populated userId object
  final FileAttachment? file;
  final DateTime createdAt;

  ChatMessage({...});

  factory ChatMessage.fromJson(Map<String, dynamic> json) {
    // userId may be a String OR a populated object
    String userId;
    String? userName;

    if (json['userId'] is Map) {
      userId = json['userId']['_id'];
      userName = json['userId']['name'];
    } else {
      userId = json['userId'] ?? '';
      userName = null;
    }

    return ChatMessage(
      id: json['id'] ?? json['_id'],
      content: json['content'] ?? '',
      room: json['room'],
      userId: userId,
      userName: userName,
      file: json['file'] != null && json['file'].isNotEmpty
          ? FileAttachment.fromJson(json['file'])
          : null,
      createdAt: DateTime.parse(json['createdAt']),
    );
  }
}
```

SocketMessage (from `message` event)

```
{
  "userName": "john",
  "text": "Hello!",
  "time": "09:12",
  "file": { "url": "...", "publicId": "...", "type": "image", "name": "photo.png" }
}
```

---

5. Complete User Flow



6. Error Handling & Edge Cases

6.1 `userId` is `null` in message history

Some messages in the database have `userId: null` due to a race condition (user disconnects before the message is saved). Handle in Flutter:

```
final userName = msg['userId']?['name'] ?? 'Unknown';
```

6.2 Empty file object `{}`

Some old messages have `"file": {}` (empty map) instead of `null`. Check:

```
final file = msg['file'];
if (file != null && file is Map && file.isNotEmpty) {
  // Has file attachment
} else {
  // No file
}
```

6.3 Socket disconnection

Implement reconnection handling:

```
socket.on('disconnect', (_) {
  // Show "Reconnecting..." overlay
});

socket.on('reconnect', (_) {
  // Re-emit joinRoom and refetch messages
  socket.emit('joinRoom', {'userName': name, 'room': room});
  fetchMessages();
});
```

6.4 File upload fails

Cloudinary may reject very large files, unsupported formats, or network timeouts. Always wrap in try/catch:

```
try {
  final file = await uploadFile(localFile);
  // Emit chatMessage with file
} catch (e) {
  // Show error toast
}
```

6.5 No `video` type support

The current server maps `video/*` MIME types to `"file"` (not `"video"`). If your app needs video display, either:

- Treat `type == "file"` with a video extension as a video, or
- Modify the server to add `"video"` detection

---

7. Flutter Quick Start

Dependencies

```
dependencies:
  socket_io_client: ^3.0.2
  http: ^1.4.0
```

Chat Service Example

```
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;
import 'package:socket_io_client/socket_io_client.dart' as IO;

class ChatService {
  static const String baseUrl = 'https://nd1cgptf-3002.uks1.devtunnels.ms';

  late final IO.Socket socket;
  String? currentUserName;
  String? currentRoom;

  // ── REST ──

  Future<User> joinRoom(String name, String room) async {
    currentUserName = name;
    currentRoom = room;
```

```
  final response = await http.post(
    Uri.parse('$baseUrl/go-to-room'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'name': name, 'room': room}),
  );

  return User.fromJson(jsonDecode(response.body)['data']);
}

Future<List<ChatMessage>> getMessages(String room) async {
  final response = await http.get(
    Uri.parse('$baseUrl/all-messages-for-room?room=$room'),
  );

  final List data = jsonDecode(response.body)['data'];
  return data.map((e) => ChatMessage.fromJson(e)).toList();
}

Future<FileAttachment> uploadFile(File file) async {
  final request = http.MultipartRequest(
    'POST',
    Uri.parse('$baseUrl/upload-file'),
  );
  request.files.add(await http.MultipartFile.fromPath('file', file.path));
  final streamedResponse = await request.send();
  final response = await http.Response.fromStream(streamedResponse);
  return FileAttachment.fromJson(jsonDecode(response.body));
}

// ── Socket.IO ──

void connect() {
  socket = IO.io(baseUrl, <String, dynamic>{
    'transports': ['websocket'],
    'autoConnect': true,
  });

  socket.onConnect((_) {
    print('Connected');
    socket.emit('joinRoom', {
      'userName': currentUserName,
      'room': currentRoom,
    });
  });

  socket.on('message', (data) {
    // Handle incoming message
    final userName = data['userName'] as String;
    final text = data['text'] as String;
    final file = data['file'] != null ? FileAttachment.fromJson(data['file']) : null;
    print('Message from $userName: $text');
  });

  socket.on('roomUsers', (data) {
    // Update user list
  });

  socket.on('displayTyping', (data) {
    // Show/hide typing indicator
  });
}

void sendMessage(String text, {FileAttachment? file}) {
  socket.emit('chatMessage', {
    'text': text,
    'file': file != null ? {
      'url': file.url,
      'publicId': file.publicId,
      'type': file.type,
```

```
        'name': file.name,
      } : null,
    });
  }

  void sendTyping(bool isTyping) {
    socket.emit('typing', isTyping);
  }

  void disconnect() {
    socket.disconnect();
  }
}
```